

Neural Global Reasoning over Relational Structures

ABSTRACT

Developing models capable of complex, multi-step reasoning is a central goal in artificial intelligence. While representing problems as graphs is a powerful approach, Graph Neural Networks (GNNs) are fundamentally constrained by their message-passing mechanism, which imposes a local bottleneck that limits global, holistic reasoning. We argue that dynamic programming (DP), which solves problems by iteratively refining a global state, offers a more powerful and suitable learning paradigm. We introduce FloydNet, a new architecture that embodies this principle. In contrast to local message passing, FloydNet maintains a global, all-pairs relationship tensor and learns a generalized DP operator to progressively refine it. This enables the model to develop a task-specific relational calculus, providing a principled framework for capturing long-range dependencies. [Theoretically, we prove that FloydNet achieves 3-WL $\setminus$ (2-FWL) expressive power, and its generalized form aligns with the k-FWL hierarchy.](#) FloydNet demonstrates state-of-the-art performance across challenging domains: it achieves near-perfect scores (often >99%) on the CLRS-30 algorithmic benchmark, finds exact optimal solutions for the general Traveling Salesman Problem (TSP) at rates significantly exceeding strong heuristics, and empirically matches the 3-WL test on the BREC benchmark. Our results establish this learned, DP-style refinement as a powerful and practical alternative to message passing for high-level graph reasoning.

1 INTRODUCTION

The development of intelligent models capable of reasoning is a great challenge in the field. Modern approaches can be analyzed through the lens of dual process theory (Kahneman, 2011), which distinguishes between intuitive "System 1" thinking and deliberate, step-by-step "System 2" thinking. While Large Language Models (LLMs) (Google, 2025; OpenAI, 2025; Deepseek, 2025) exhibit impressive emergent reasoning akin to System 1 (Wang & Zhou, 2024), ensuring structured and verifiable logic, the hallmark of System 2, remains an open problem. A key step towards building such reasoners is learning to model the complex, global relationships between representations, not just their individual features.

Neural Algorithmic Reasoning (NAR) (Veličković & Blundell, 2021) provides a promising framework for this challenge, aiming to bridge the logic of classical algorithms with the power of deep learning. Graph Neural Networks (GNNs), particularly the Message Passing (MPNN) paradigm (Gilmer et al., 2017), have become a cornerstone of NAR. However, this architectural choice introduces a fundamental bottleneck: local message passing is inherently insufficient for problems demanding global, holistic reasoning. This locality manifests in two key limitations: **restricted information flow**, where capturing long-range dependencies requires deep GNNs that are prone to over-smoothing and over-squashing (Li et al., 2018; Topping et al., 2022), hindering effective scaling; and **limited expressive power**, with most MPNNs being bounded by the 1-WL test (Xu et al., 2019). Although graph transformers (GTs) (Dwivedi & Bresson, 2021; Ying et al., 2021) offer global receptive fields, they often rely on complex positional or structural encodings (Ying et al., 2021; Rampásek et al., 2022) to inject inductive bias into the graph, which can be difficult to design and may still underuse the higher-order relational structure of the graph.

In this work, we propose a new paradigm inspired by Dynamic Programming (DP). Instead of enriching node features locally, we learn a global refinement operator that acts directly on a representation of all-pairs relationships. The classic Floyd–Warshall (FW) algorithm exemplifies this principle: it

[Diagram of the FloydBlock architecture. An input tensor R of shape (N, N, d_r) is processed through three parallel Linear layers to produce Query (Q_{ik}), Key (K_{ijk}), and Value (V_{ijk}) tensors. The Key and Value tensors are then processed through a Pivotal Attention mechanism (indicated by circles with dots) to produce intermediate tensors K_{ijk} and V_{ijk} . These are then processed through another Linear layer to produce the final output O_{ik} of shape (N, N, d_r) .]

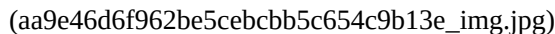


Diagram of the FloydBlock architecture. An input tensor R of shape (N, N, d_r) is processed through three parallel Linear layers to produce Query (Q_{ik}), Key (K_{ijk}), and Value (V_{ijk}) tensors. The Key and Value tensors are then processed through a Pivotal Attention mechanism (indicated by circles with dots) to produce intermediate tensors K_{ijk} and V_{ijk} .

and V_{ijk} . These are then processed through another Linear layer to produce the final output O_{ik} of shape (N, N, d_r) .

```

...
def PivotAttnCore(q_ik, k_ijk, k_jk, v_ijk, v_jk):
    # All input tensors of shape [b,n,n,h,d_h]
    # come from different linear transform of R

    k_ijk = k_ijk[:, :, :, None, :, :, :] +
        k_jk[:, :, None, :, :, :, :]
    v_ijk = v_ijk[:, :, :, None, :, :, :] +
        v_jk[:, :, None, :, :, :, :]
    q_ikx = q_ik[:, :, :, None, :, :, :]

    a_ijk = torch.sum(q_ikx * k_ijk, dim=-1)
    d = q_ik.shape[-1] # per head embed dim
    a_ijk = a_ijk / (d ** 0.5) # [b,n,n,h]
    v_ijk = torch.softmax(a_ijk, dim=-3) # dim j

    o_ijk = v_ijk * w_ijk[:, :, :, :, :, None]
    o_ik = torch.sum(o_ijk, dim=-4) # dim j
    return o_ik # [b,n,n,h,d_h], same as input
...

```

Figure 1: **FloydBlock** and **Pivotal Attention Mechanism**. Each **FloydBlock** updates the representation for a given pair (i, k) by using its state as a query to attend to all possible two-hop paths that pass through an intermediate "pivot" node j .

updates all-pairs shortest paths by systematically considering each node j as a potential intermediate pivot in the path from i to k . This iterative, all-pairs update allows information to propagate globally and interact in a structured way. Our key insight is to generalize this algorithmic structure. We replace the fixed update rule of FW (e.g., $D_{ik} \leftarrow \min_j (D_{ij} + D_{jk})$) with a learned, high-dimensional neural operator, creating an architecture that is algorithmically aligned with the global reasoning pattern of DP.

This principle is embodied in our new architecture, **FloydNet**, which maintains and refines a global pairwise relationship tensor $R \in \mathbb{R}^{N \times N \times d_r}$. Its core component, the **FloydBlock**, acts as a *learned, generalized DP operator*. For each pair (i, k) , it updates its representation R_{ik} by aggregating information from all relational paths of the form $i \rightsquigarrow j \rightsquigarrow k$, mediated by every pivot node j . This update is not a fixed operation but a flexible transformation learned by our proposed **Pivotal Attention** mechanism. This data-driven approach allows **FloydNet** to learn a task-specific "relational calculus" and discover effective reasoning strategies beyond predefined heuristics.

Our primary contributions are:

1. We propose a new paradigm for graph reasoning that shifts from local message passing to a global, DP-inspired refinement of all-pairs relationships.
2. We introduce **FloydNet**, a novel architecture that realizes this paradigm through our proposed **Pivotal Attention** mechanism. We have also developed a highly optimized compute kernel of **Pivotal Attention**, which is efficient in processing large-scale graphs.
3. We provide extensive empirical validation, showing that **FloydNet** establishes new state-of-the-art results across a diverse range of challenging benchmarks, from algorithmic reasoning to combinatorial optimization.

2 THE FLOYDNET ARCHITECTURE

At its core, FloydNet is designed to implement the principle of learning a generalized Dynamic Programming operator. Instead of operating on node features, it maintains and iteratively refines a global pairwise relationship representation, $\mathbf{R} \in \mathbb{R}^{N \times N \times d_r}$. As illustrated in Figure 2, the architecture consists of three main stages: initialization of the relationship tensor, iterative refinement through a stack of FloydBlock layers, and a final decoder for task-specific representation learning.

2.1 INITIALIZATION OF THE PAIRWISE RELATIONSHIP REPRESENTATION

The initial all-pairs relationship tensor, $\mathbf{R}^{(0)}$, is constructed by integrating node, edge, and graph-level features into a unified relational representation. Given node features $\mathbf{X} \in \mathbb{R}^{N \times d_n}$, optional edge features $\mathbf{E} \in \mathbb{R}^{N \times N \times d_e}$, and optional graph features $\mathbf{G} \in \mathbb{R}^{d_g}$, we initialize the representation for each pair (i, j) as:

$$\mathbf{R}_{ij}^{(0)} = \text{MLP}_{\{\text{init}\}}(\text{left}(\text{left}(\mathbf{G}, \mathbf{X}_i, \mathbf{X}_j, \mathbf{E}_{ij})\text{right}) \quad (1)$$

where $[\cdot, \cdot]$ denotes concatenation.

2.2 ADAPTING FOR NODE AND GRAPH-LEVEL TASKS VIA A SUPERNODE

To handle node and graph-level tasks, we introduce a *learnable SuperNode* (SN), a virtual node that acts as a global information aggregator.

- For Node-Level Tasks, the feature for node i is its learned relationship with the SuperNode: $\mathbf{h}_i = \mathbf{R}_{i, \text{SN}}^{(L)}$.
- For Graph-Level Tasks, the graph representation is the SuperNode's self-relationship: $\mathbf{h}_{\{\text{graph}\}} = \mathbf{R}_{\{\text{SN}\}, \{\text{SN}\}}^{(L)}$.

As illustrated in Figure 2B, this provides a unified mechanism, allowing the model to learn optimal, task-specific aggregation functions through the same Pivotal Attention mechanism. This is achieved with negligible additional cost, increasing the graph size N by only 1.

2.3 THE FLOYDBLOCK: A LEARNED DP OPERATOR

The central component of FloydNet is the `FloydBlock`, as shown in Figure 2. It takes a relationship tensor $\mathbf{R}^{(l-1)}$ and produces a refined tensor $\mathbf{R}^{(l)}$. A stack of L such blocks is used to learn relational representation progressively. Each block follows a standard pre-LN Transformer structure, consisting of our Pivotal Attention module and a Feed-Forward Network (FFN):

$$\mathbf{R}' = \mathbf{R}^{(l-1)} + \text{PivotalAttention}(\text{Norm}(\mathbf{R}^{(l-1)})) \quad (2)$$

$$\mathbf{R}^{(l)} = \mathbf{R}' + \text{FFN}(\text{Norm}(\mathbf{R}')) \quad (3)$$

where `Norm` can be LayerNorm, RMSNorm (Zhang & Sennrich, 2019), or QK-Norm (Dehghani et al., 2023).

2.3.1 PIVOTAL ATTENTION: THE CORE MECHANISM

The Pivotal Attention mechanism, detailed in Figure 1, is the core of the `FloydBlock`. It updates the relationship representation for a target pair (i, k) by aggregating information from all possible two-hop relational paths of the form $i \rightsquigarrow j \rightsquigarrow k$, mediated by every possible "pivot" node $j \in \mathcal{V}$.

For a given target pair (i, k) , its representation $\mathbf{R}'_{ik}$ is projected to form a **Query** vector,

$\mathbf{q}_{ik}$. For each potential pivot j , the representations of the path segments, $\mathbf{R}'_{ij}$ and $\mathbf{R}'_{jk}$, are projected to form **Key** and **Value** vectors ($\mathbf{k}_{ij}$, $\mathbf{v}_{ij}$ and $\mathbf{k}_{jk}$, $\mathbf{v}_{jk}$). **Key** and **Value** vectors on the same path are combined by an operation $\mathcal{C}$ (e.g., addition or multiplication). Then, standard attention is applied:

$$\mathbf{k}_{ijk} = \mathcal{C}(\mathbf{k}_{ij}, \mathbf{k}_{jk}), \quad \mathbf{v}_{ijk} = \mathcal{C}(\mathbf{v}_{ij}, \mathbf{v}_{jk}) \quad (4)$$

$$\mathbf{o}_{ik} = \text{MultiheadAttention}_j(\mathbf{q}_{ik}, \mathbf{k}_{ijk}, \mathbf{v}_{ijk}) \quad (5)$$

For most graph tasks, an additive combine operation is suitable. The multiplicative operation may be useful for some geometric tasks (see Appendix B.1). The overall computational complexity of Pivotal Attention is $\mathcal{O}(\tilde{N}^3 \cdot d_r + N^2 \cdot d_r^2)$ (see Appendix B.4). A naive implementation of Pivotal Attention would create large intermediate tensors $\mathbf{k}_{ijk}$ and $\mathbf{v}_{ijk}$ of size $\mathcal{O}(N^3 \cdot d_r)$, which is memory-intensive for large graphs. To address this, we implemented a dedicated compute kernel that avoids storing these large intermediate tensors, reducing memory usage to $\mathcal{O}(N^2 \cdot d_r)$. While this optimization does not alter the theoretical computational cost, it improved training performance by more than 20 times in practice, as hardware memory bandwidth is a limiting factor.

2.4 PERMUTATION EQUIVARIANCE AND POSITIONAL ENCODINGS

FloydNet is inherently permutation equivariant and does not require Positional Encodings (PEs). Its core operation is fundamentally graph-structured. Permuting the node indices induces an equivalent permutation on the relationship tensor $\mathbf{R}$, and the symmetric aggregation over all pivots naturally preserves this equivariance.

2.5 GENERAL FORM OF FLOYDNET (K-FLOYDNET)

While a standard graph edge connects two vertices, a hypergraph edge can connect any number of vertices. The k-FloydNet is the general form of FloydNet that works on a hypergraph, updating hyper-edges $\mathbf{e} = (v_1, v_2, \dots, v_k) \in V^{\times k}$ that connect to k vertices, and uses a pivot $p \in V$ to enumerate all neighboring hyper-edges $\mathbf{e}'$ that differ from $\mathbf{e}$ by vertex p . The linear projection of embedding of hyper-edges $\mathbf{e}$ generated from the same pivot p are combined using operator $\mathcal{C}$ to form **Key** and **Value** vectors, which correspond to **Query** vectors projected from the embeddings of hyper-edges $\mathbf{e}$. We use notation $\mathbf{e}[i] \leftarrow p$ to represent the substitution of the i -th vertex in hyper-edge $\mathbf{e}$ with the pivot p , thereby constructing a neighboring hyper-edge $\mathbf{e}' = (v_1, v_2, \dots, v_{i-1}, p, v_{i+1}, \dots, v_{k-1}, v_k)$. The formula of Pivotal Attention of k-FloydNet is:

$$\mathbf{k}_{\mathbf{e},p} = \mathcal{C}(\mathbf{k}_{\mathbf{e}[1] \leftarrow p}, \mathbf{k}_{\mathbf{e}[2] \leftarrow p}, \dots, \mathbf{k}_{\mathbf{e}[k] \leftarrow p}) \quad (6)$$

$$\mathbf{v}_{\mathbf{e},p} = \mathcal{C}(\mathbf{v}_{\mathbf{e}[1] \leftarrow p}, \mathbf{v}_{\mathbf{e}[2] \leftarrow p}, \dots, \mathbf{v}_{\mathbf{e}[k] \leftarrow p}) \quad (7)$$

$$\mathbf{o}_{\mathbf{e}} = \text{MultiheadAttention}_p(\mathbf{q}_{\mathbf{e}}, \mathbf{k}_{\mathbf{e},p}, \mathbf{v}_{\mathbf{e},p}) \quad (8)$$

FloydNet is the special case where $k = 2$, and self-attention of the Transformer is the special case where $k=1$. The expressive power of k-FloydNet increases with the value of k , as higher-order interactions among sets of vertices can be captured more effectively. The overall computational complexity of Pivotal Attention is $\mathcal{O}(N^{k+1} \cdot d_r + N^k \cdot d_r^2)$ (see Appendix B.3 and Appendix B.4).

[Figure 2: The FloydNet framework. (A) shows the model transforming various graph inputs ($k=2, k=3, k=4$) into a unified relational representation $\mathbf{R}$. (B) shows the SuperNode providing a unified representation for graph, node, and

edge information within R .](7a207f7bb5a20f849aee089439e24a84_img.jpg)

Figure 2 illustrates the FloydNet framework. Part (A) shows the model transforming various graph inputs into a unified relational representation R . The inputs are graphs with hyperedges of size $k=2$, $k=3$, and $k=4$. Each graph is processed through an encoder that includes an 'Initiation' block, a 'Norm' block, a 'Pivotal Attention' block, another 'Norm' block, a 'Feed Forward' block, and a 'Decoder' block. The output of the encoder is a unified relational representation R . Part (B) shows the SuperNode providing a unified representation for graph, node, and edge information within R . The SuperNode is a graph with a central node connected to all other nodes, and it is processed through the same encoder structure to produce the final relational representation R .

Figure 2: The FloydNet framework. (A) shows the model transforming various graph inputs ($k=2$, $k=3$, $k=4$) into a unified relational representation R . (B) shows the SuperNode providing a unified representation for graph, node, and edge information within R .

Figure 2: **The FloydNet framework.** (A) The model transforms various graph inputs into a unified relational representation R . (B) The SuperNode provides a unified representation for graph, node, and edge information within R .

3 THEORETICAL ANALYSIS OF K-FLOYDNET

We now formally analyze the expressive power of k -FloydNet. We leverage the connection between our architecture and the Weisfeiler-Lehman (WL) hierarchy.

Theorem 1 (k -FloydNet is equivalent to k -FWL). *The k -FloydNet architecture functionally implements the k -Folklore Weisfeiler-Lehman (k -FWL) color refinement algorithm. Consequently, it can distinguish any pair of non-isomorphic graphs distinguishable by the k -FWL test.*

Proof Sketch. The k -FWL test refines the color of a k -tuple based on the multiset of colors of its neighbors (formed by swapping one node with a pivot). Pivotal Attention (Section 2) performs exactly this operation in a continuous embedding space: it aggregates information from all neighbor tuples generated by a pivot p . Since the attention mechanism is permutation-invariant with respect to p and can model arbitrary non-linear hashing functions (via MLPs), it satisfies the conditions of the k -FWL update rule. (See Appendix C for full proof). $\square$

Corollary (3-WL Expressiveness): Since FloydNet corresponds to 2-FloydNet, and it is established that 2-FWL is equivalent to 3-WL (Cai et al., 1992), FloydNet achieves **3-WL expressive power**. This strictly surpasses the 1-WL limit of standard MPNNs.

Theorem 2 (Principled Long-Range Information Propagation). *After L layers, the representation $\mathbf{R}_{ik}^{(L)}$ in FloydNet integrates information from all relational paths of length up to 2^L between nodes i and k .*

Proof Sketch. By induction. For $L = 1$, $\mathbf{R}_{ik}^{(1)}$ aggregates paths $i \rightsquigarrow j \rightsquigarrow k$ (length 2). If layer $L - 1$ captures length 2^{L-1} , then combining two such segments via a pivot concatenates the paths, covering length $2^{L-1} + 2^{L-1} = 2^L$. This exponential growth in receptive field mitigates the over-squashing problem common in MPNNs. $\square$

4 EXPERIMENTS

To validate the design of FloydNet, we conduct a comprehensive set of experiments designed to answer three key questions: (1) Does it possess the theoretical expressive power to overcome the limitations of current GNNs? (2) Can it learn complex, global algorithmic reasoning procedures? (3) Does this capability translate to strong performance on challenging real-world problems? We address these questions by evaluating our model on graph property prediction, neural algorithmic reasoning, Traveling Salesman Problem and LRGB.

Our code, data generation scripts, and exact hyperparameters are available at [http://anonymous__for_submission] (http://anonymous_for_submission). Where applicable (e.g., CLRS), we use the official evaluation harness without hints at test time unless explicitly stated; for TSP we guarantee train/test isolation across sizes and random seeds and retain only instances with unique optima to avoid solution multiplicity confounds.

4.1 EXPRESSIVE POWER: GRAPH PROPERTY PREDICTION

We first measure FloydNet’s ability to overcome the limitations of standard GNNs on two complementary fronts: quantitative structure counting and qualitative isomorphism testing.

****Quantitative Expressiveness on Homomorphism Counting Tasks.**** To probe the model’s ability to count complex graph structures, we use the synthetic property prediction tasks from Zhang et al. (2024a). These benchmarks, which involve homomorphism and subgraph isomorphism counting, act as a practical barometer for transcending the 1-WL test. As shown in Tables 1 and 3, while standard MPNNs fail on most tasks, FloydNet consistently reduces the Mean Absolute Error to near-zero across all graph, node, and edge-level tasks. This strong quantitative performance provides clear initial evidence that FloydNet operates well beyond the 1-WL limit.

Table 1: MAE on homomorphism counting. FloydNet encodes all substructures and achieves near-zero error, substantially outperforming all baselines.

Model \ Task	Graph-level	Node-level	Edge-level		
	- - - - - - - - - -				
					
					
MPNN (GIN)	.300	.233	.254	.505	.478
Subgraph GNN	.011	.015	.012	.004	.058
Local 2-GNN	.008	.008	.010	.003	.004
Local 2-FGNN	.003	.005	.004	.005	.007
FloydNet	.000	.001	.001	.000	.000

Table 2: Accuracy on the BREC benchmark. k-FloydNet’s distinguishing power is empirically identical to the k-FWL test.

Model / Heuristic	Accuracy (%)
MPNN (1-WL equiv.)	<20%
Graphormer	19.8%
PPGT	58.5%
KP-GNN	68.8%
3-WL(2-FWL) Test	67.5%
FloydNet	67.5%
3-FloydNet	95.0%
4-FloydNet	99.8%

Table 3: MAE on the (Chordal) Cycle Counting task. FloydNet accurately counts all types of cycles and substantially outperforms all baselines.

	Graph-level				Node-level				Edge-level			
Model \ Task												
- - - - - - - - - - - - - - - -												
Image: triangle icon					Image: square icon				Image: pentagon icon			
icon</p> <p>Image: heptagon icon</p>					icon</p> <p>Image: octagon icon</p>				icon Image:			

square icon |  |  |  |  |  |  |  |  |  |  |
 | MPNN | .358 | .208 | .188 | .146 | .261 | .205 | .600 | .413 | .300 | .207 | .318 | .237 | - | - | - | - | - |
 | Subgraph GNN | .010 | .020 | .024 | .046 | .007 | .027 | .003 | .005 | .092 | .082 | .050 | .073 | .001 | .003 | .090 | .096 |
 .038 | .065 |
 | Local 2-GNN | .008 | .011 | .017 | .034 | .007 | .016 | .002 | .005 | .010 | .023 | .004 | .015 | .001 | .005 | .010 | .019 | .005 |
 .014 |
 | Local 2-FGNN | .003 | .004 | .010 | .020 | .003 | .010 | .004 | .006 | .012 | .021 | .004 | .014 | .003 | .006 | .012 | .022 | .005 |
 .012 |
 | **FloydNet** | .000 | .000 | .001 | .001 | .000 | .001 | .000 | .000 | .000 | .000 | .000 | .000 | .000 | .000 | .000 | .000 |
 .000 | .000 |

****Qualitative Expressiveness on Isomorphism Testing.**** To precisely characterize FloydNet’s power in distinguishing non-isomorphic graphs, we evaluate it on the challenging BREC benchmark (Wang & Zhang, 2024). The results, summarized in Table 2, lead to a key finding: FloydNet’s performance is identical to the theoretical 3-WL heuristic.

****Expressiveness of k-FloydNet.**** We further empirically validate our theoretical claims regarding k-FloydNet. The results, summarized in Table 2 and detailed in Appendix D.1, confirm that the architecture demonstrates scalable expressive power consistent with the theory. As shown in Table 2, 3-FloydNet and 4-FloydNet achieve accuracies of 95.0% and 99.8% respectively on BREC. These results align perfectly with our theoretical analysis in Section 3, confirming that the practical distinguishing power of the k-FloydNet model matches the theoretical k-FWL hierarchy.

4.2 NEURAL ALGORITHMIC REASONING

We now evaluate FloydNet’s core capability: learning algorithmic computations.

****The CLRS Benchmark.**** The CLRS-30 benchmark (Veličković et al., 2022) is a standard suite for assessing algorithmic reasoning. We follow the standard CLRS-30 protocol, training FloydNet model for 30 algorithmic tasks. Evaluation is performed on out-of-distribution instances with larger problem sizes, without providing any ground-truth hints to the model at test time. We evaluate on the full benchmark and report aggregated performance across eight categories in Table 4. The results show that FloydNet consistently and significantly outperforms previous state-of-the-art models. Notably, it achieves near-perfect scores (> 95%) across most of the algorithms, including those that are difficult for standard GNNs, such as Strings, Graph algorithms (e.g., Floyd-Warshall), and Sorting. We also found that intermediate result (hints) are not as beneficial as expected: for most algorithms, using hints can actually degrade performance. To rigorously test length scalability, we extended the evaluation protocol to problem sizes up to $N = 256$, following the scale of the SALSA-CLRS benchmark (Minder et al., 2023). As visualized in Figure 3(a), FloydNet maintains robust performance at this scale, with the no-hint model showing superior extrapolation capabilities. Full per-algorithm results and detailed extrapolation scores are available in Appendix D.2.

!Figure 3: Probing the Scalability and Reasoning Logic of FloydNet on CLRS. (a) OOD Scalability Analysis (n -> 256). (b) Algorithmic Alignment.](73c3e4508cae529acf4e6c7fa70b361a_img.jpg)

Figure 3 consists of two parts. Part (a) shows two line graphs of 'Test Performance (No Hints)' and 'Test Performance (With Hints)' for the Maximum Subarray problem. The x-axis is 'Number of Nodes' (64 to 256) and the y-axis is 'In-Sample' performance (0% to 100%). Four models are compared: Find Max Subarray (orange circles), MST Prim (green squares), Dijkstra (blue diamonds), and BFS (pink triangles). In the 'No Hints' graph, all models maintain high performance. In the 'With Hints' graph, performance drops for all models as the number of nodes increases, with Find Max Subarray showing the steepest decline. Part (b) illustrates the 'Algorithmic Alignment' for step 5 and step 6 of Kadane’s algorithm. It shows the 'current maximum subarray' (step 5) and the 'new predicted maximum subarray' (step 6) as a sequence of numbers (-0.6, 0.7, 0.2, -0.3, 0.4, 0.1) with a 'Model' block in the center. Arrows indicate the model’s prediction of the next number (0.1) and the updated maximum subarray.

Figure 3: Probing the Scalability and Reasoning Logic of FloydNet on CLRS. (a) OOD Scalability Analysis ($n \rightarrow 256$). (b) Algorithmic Alignment.

Figure 3: **Probing the Scalability and Reasoning Logic of FloydNet on CLRS.** (a) We extend the OOD evaluation to problem sizes up to $n = 256$, following the protocol of Minder et al. (2023). The results indicate that models trained *without hints* (upper) generally exhibit superior length extrapolation capabilities compared to those trained *with hints* (lower). (b) Visualization of the internal reasoning process for the Maximum Subarray problem (Kadane’s algorithm). The model iteratively predicts the new maximum subarray based on the current maximum subarray and the next number corresponding to the current step. The clear alignment demonstrates that FloydNet has successfully learned to emulate the algorithm’s state-update logic.

Probing Algorithmic Reasoning. To verify that FloydNet learns the underlying algorithmic process, we probed its step-by-step reasoning on dynamic programming tasks. We use *Find Max. Subarray* (Kadane’s algorithm) as a case study, where the correct solution requires maintaining and updating the optimal subarray at each step of a traversal. As visualized in Figure 3b, FloydNet’s strong alignment with intermediate trajectory demonstrates that FloydNet is not merely memorizing input-output pairs but has successfully learned to emulate the core, state-updating logic of the algorithm.

Table 4: Average test accuracy (%) on the CLRS-30 benchmark, aggregated by algorithm category. N denotes the number of algorithms in each group. All models are trained on problem sizes of $n = 16$ and tested on larger instances of $n = 64$.

Class	Model	Sort (N=4)	Search (N=3)	D&C (N=1)	Greedy (N=2)	DP (N=3)	Graph (N=12)	String (N=2)	Geometry (N=3)	Total (N=30)
	Triplet-GMPNN (Ibarz et al., 2022)	75.64	58.83	76.36	91.22	81.99	86.42	49.09	94.09	80.04
	GNN RANR (Xu & Veličković, 2024)	94.15	82.90	83.53	91.66	42.69	74.15	49.12	88.44	75.78
	G-ForgetNet (Bohde et al., 2024)	78.09	63.84	78.97	91.79	86.70	88.80	54.73	95.09	82.89
	GT									
	RT (Diao & Loynd, 2022)	50.01	65.31	66.52	85.33	83.20	65.33	32.52	84.55	66.18
	ET (Müller et al., 2024)	82.26	63.00	64.44	81.67	83.49	86.08	54.84	88.22	80.13
	Ours FloydNet	100.00	91.60	86.20	93.17	90.02	98.60	99.72	99.68	96.64

4.3 PUSHING THE LIMITS: THE TRAVELING SALESMAN PROBLEM (TSP).

To test our model on a canonical NP-hard problem, we evaluate it on the Traveling Salesman Problem (TSP). The difficulty stems from the factorial growth in potential solutions, making the search for the shortest tour computationally intractable for all but the smallest instances.

Settings and Training. We constructed two new benchmarks for this task. The non-metric TSP dataset is composed of random complete graphs with up to 200 nodes, where edge weights are integers uniformly sampled from $[1, 100]$. The metric TSP dataset is generated by uniformly sampling N unique integer coordinates (X, Y) from the range $[1, 100]$ in 2D space, with edge weights defined as the Euclidean distances between coordinates. Since standard TSP instances often possess multiple optimal solutions, training a deterministic regressor can lead to mode averaging. To address this, we formulate the training within a Denoising Diffusion Probabilistic Model (DDPM) framework (Ho et al., 2020). This allows FloydNet to learn the underlying distribution of optimal tours, naturally handling instances with multiple valid solutions. To ensure a fair and unambiguous evaluation, we use the state-of-the-art exact solver, Concorde (Applegate et al., 1998; Aldous & Percus, 2003), to find the ground-truth optimal tour. Our model is trained exclusively

on smaller graphs ($N \leq 100$) and tested on larger, unseen graphs ($100 < N \leq 200$) to assess its out-of-distribution generalization. We benchmark FloydNet against Linkern, a highly-optimized and widely-used heuristic solver based on the Chained Lin-Kernighan algorithm (Lin & Kernighan, 1973). To further analyze the capability of FloydNet, we constructed subsets of both the Metric and Non-Metric datasets by retaining only those instances possessing a unique optimum. Subsequently, we evaluate our model on these single-solution datasets. A detailed description of the problem, solvers, and our data generation pipeline is provided in Appendix D.3.

****Result.**** On held-out general TSP instances with $100 < N \leq 200$, our model demonstrates a strong ability to significantly improve the optimality rate through multiple sampling as shown in Figure 4a. Specifically, it increases the optimality rate from 83.6% to 99.8% after 10 samples, substantially surpassing the 38.8% achieved by the strong Linkern heuristic. Furthermore, it exhibits excellent extrapolation capabilities concerning the number of nodes (N). As N increases, the optimality rate of Linkern rapidly deteriorates, dropping to 16.1% for instances where $180 \leq N \leq 200$. In sharp contrast, our model with 10 samples maintains a robust optimality rate of 99.4% in this challenging range. We also examined a subset containing only single-solution instances. On non metric TSP instances with $100 < N \leq 200$, the FloydNet finds the exact optimal tour in 92.6% of cases, as shown in Figure 4b. This performance markedly outperforms the Linkern heuristic, which achieves only a 15.7% optimality rate on these larger instances. The performance gap widens significantly with instance size: Linkern’s optimality rate plummets to 1.3% when $180 \leq N \leq 200$, while FloydNet still achieves 88.3%. For smaller validation instances ($N \leq 100$), FloydNet demonstrates robust performance within the training distribution, with an optimality rate exceeding 96%. On Metric TSP instances, the Linkern heuristic performs substantially better than on Non-Metric TSP instances, leveraging the underlying geometric properties. In this setting, FloydNet achieves an optimality rate that is comparable to that of the Linkern heuristic.

****Constraint and error analysis.**** We analyzed the failure cases of FloydNet on single-solution instances and found that only a small fraction (0.8%) corresponded to valid but suboptimal TSP tours, 6.2% produced multiple disjoint cycles covering all nodes, and the remaining 93% generated fewer edges than nodes, resulting in incomplete tours that failed to visit all nodes. We hypothesize that this phenomenon may stem from numerical issues or interactions with mechanisms such as initialization, normalization, or optimization. Notably, scaling up the model effectively mitigates the effect, highlighting a promising direction for future study.

****Scaling Properties.**** As shown in Figures 4d and 4e, performance consistently improves with both larger training sets and increased model capacity. To substantiate this observation, we conducted two sets of controlled experiments with FloydNet on single-solution instances. In the first, we fixed the training set size and varied the model capacity from $L = 1$ to $L = 96$ layers, with larger values infeasible due to memory constraints; we also fixed the model capacity and varied the number of training samples. In both cases, we evaluated performance on out-of-distribution test instances to assess generalization. All experiments were trained under identical hyperparameter settings and terminated after a total of 2.5M training samples. The results demonstrate that FloydNet benefits from both data and model size scaling, in contrast to many traditional MPNNs whose performance typically deteriorates with depth due to over-smoothing. Thanks to its global refinement mechanism, FloydNet avoids this pitfall and establishes itself as a scalable architecture, where increasing model size and data directly translates into stronger reasoning capabilities.

![[Figure 4: Analysis of FloydNet on the TSP. (a) Full Benchmark (Multi-Solution) showing Optimality Rate vs Number of Nodes for Metric TSP and Non-Metric TSP. (b) Algorithmic Imitation (Single-Solution) showing Optimality Rate vs Number of Nodes. (c) Failure Case Analysis scatter plot. (d) Data Scaling Analysis showing Optimality Rate vs Training Dataset Size. (e) Model Scaling Analysis showing Optimality Rate vs Model Parameters.]]

(b93cbfb52e37619e688175a6aad9edd9_img.jpg)

Figure 4: Analysis of FloydNet on the TSP. (a) Full Benchmark (Multi-Solution) showing Optimality Rate vs Number of Nodes for Metric TSP and Non-Metric TSP. (b) Algorithmic Imitation (Single-Solution) showing Optimality Rate vs Number of Nodes. (c) Failure Case Analysis scatter plot. (d) Data Scaling Analysis showing Optimality Rate vs Training Dataset Size. (e) Model Scaling Analysis showing Optimality Rate vs Model Parameters.

Figure 4: ****Analysis of FloydNet on the TSP.**** (a) Optimality rate on the benchmark (including multi-solution

instances). The yellow region denotes validation on in-distribution sizes ($N \leq 100$), while the cyan region demonstrates OOD generalization to larger problem sizes ($100 < N \leq 200$). (b) Performance on the subset of single-solution instances, testing the model’s precise alignment with a unique optimal ground truth. (c) Distribution of error types. (d-e) Scaling laws showing that FloydNet effectively utilizes increased data and model depth, avoiding the over-smoothing issues typical of deep MPNNs.

4.4 REAL-WORLD APPLICATIONS

Finally, we validate FloydNet’s practical utility on real-world benchmarks where long-range interactions are critical. The results on the Long-Range Graph Benchmark (LRGB) (Dwivedi et al., 2022), shown in Table 5, highlight the strengths of our model’s relational reasoning paradigm. Without tuning, FloydNet establishes a new state-of-the-art by a significant margin on the link prediction task (PCQM-Contact) and also secures top performance on the node classification task COCO-SP and ZINC.

On the remaining graph-level and node-level tasks, FloydNet remains highly competitive with other leading models. Our analysis indicates that performance on these tasks, particularly graph property prediction, is constrained by overfitting on the training set. A detailed discussion of this phenomenon is provided in Appendix D.4.

Table 5: Mean performance on Long-Range Graph Benchmarks over five runs.

Class	Task Dataset Metric	Graph Class. Peptides-Func AP $\uparrow$	Graph Reg. Peptides-Struct MAE $\downarrow$	Node Class. PascalVOC-SP F1 $\uparrow$	Node Class. COCO-SP F1 $\uparrow$	Link Pred. PCQM-Contact MRR $\uparrow$
GNN	GCN(Brossard et al., 2020)	0.5930	0.3496	0.1268	0.0841	0.3234
GT	Exphormer(Shirzad et al., 2023)	0.6258	0.6535	0.2512	0.2500	0.3446
	GPS(Rampášek et al., 2022)	0.3724	0.3430	0.3774	0.3587	0.3437
SSM	Graph-Mamba(Wang et al., 2024)	0.6739	0.2478	0.4191	0.3960	0.3395
Ours	FloydNet	0.6224	0.2612	0.4046	0.4901	0.6143

5 RELATED WORK

5.1 ARCHITECTURES BEYOND 1-WL EXPRESSIVENESS

A central theme in GNN research is overcoming the expressive limitations of MPNNs, famously bounded by the 1-WL test (Xu et al., 2019; Morris et al., 2019). This pursuit has led to a variety of more powerful architectures, including Subgraph GNNs (Bevilacqua et al., 2022; Zhang & Li, 2021), Spectral GNNs (Zhang et al., 2024b; Lim et al., 2023) and higher-order GNNs (Maron et al., 2019b;a) grounded in more powerful theoretical tests. Among these, the k-Folklore Weisfeiler-Lehman (k-FWL) test (Cai et al., 1992; Maron et al., 2019a) serves as a key benchmark for models that reason over tuples of nodes. The 2-FWL test, for instance, refines representations for pairs of nodes (u, v) by aggregating over all intermediate pivot nodes w , considering relational triangles of the form $u \rightarrow w \rightarrow v$. Architectures like the Folklore GNN (FGNN) (Zhang et al., 2024a) are designed to mimic this process.

While many expressive models rely on enriching local structures or implementing complex tensor operations, FloydNet introduces a distinct computational paradigm. It is based on learning a global, DP-style operator that is algorithmically aligned with the pivotal aggregation pattern of the FWL tests. Our empirical results strongly support a key theoretical connection: the relational refinement in k-FloydNet is not merely a novel heuristic but is empirically equivalent in expressive power to the k-FWL test. We give a formal discussion on Appendix C. This result precisely characterizes k-FloydNet, positioning it as an elegant and practical realization of this powerful theoretical class.

5.2 TRANSFORMERS AND POSITIONAL ENCODINGS

The standard Transformer architecture (Vaswani et al., 2017) excels at processing 1D sequences. However, graphs lack a canonical ordering, making the direct application of Transformers challenging. Graph Transformers (GTs) (Dwivedi & Bresson, 2021; Ying et al., 2021; Kim et al., 2022; Rampasek et al., 2022; Lai et al., 2023) adapt this architecture by treating nodes as an unordered set, but this makes them inherently blind to the graph’s topology. To compensate, they require carefully designed Positional and Structural Encodings (PSEs) to re-inject the structural information that was lost.

This work proposes a more principled generalization. Instead of operating on a 1D set of node representations, FloydNet’s Pivotal Attention mechanism operates directly on the 2D relational space of all node pairs. This perspective sidesteps the need for PSEs entirely. While many PSEs have been developed, often based on Laplacian eigenvectors (Kreuzer et al., 2021), they introduce their own challenges, such as eigenvector ambiguity (Lim et al., 2023). Even more robust spectral features (Huang et al., 2024; Li et al., 2020) have fundamental theoretical limits; a comprehensive analysis by Zhang et al. (2024b) shows their expressive power is strictly weaker than the 3-WL test. Thus, most GTs, despite their architectural complexity, operate below the expressiveness ceiling that FloydNet empirically achieves. By building the graph’s relational structure into its core computational block, FloydNet provides an elegant and powerful alternative that is naturally permutation-equivariant.

6 CONCLUSION AND FUTURE WORK

This paper introduced FloydNet, a new paradigm for graph reasoning based on a learned, DP-style global refinement operator that replaces local message passing. Theoretically, FloydNet and its’ extends (k-FloydNet) is equivalent to the powerful 2-FWL (k-FWL) test, yet empirically, it achieves such distinguishing power on challenging benchmarks. This expressiveness translates to state-of-the-art performance on algorithmic reasoning (CLRS-30) and combinatorial optimization (TSP), establishing our approach as a powerful and practical paradigm for global graph reasoning.

Future work can focus on improving scalability beyond the current $\mathcal{O}(N^3)$ complexity through sparse mechanisms, extending the framework to new domains like multimodality for which its relational structure is a natural fit, and further strengthening its theoretical foundations by connecting to Abstract DP theory (Bertsekas, 2022; 2025). Ultimately, we envision FloydNet as a core component of future System-2 reasoning systems.

Rest of paper (reference and Appendix) is removed.